Intent Drift Benchmark: Initial Issues and Current State

A two-page brief on what the first benchmark got wrong and what the suite is now.

1. What the initial benchmark measured

The benchmark asks whether a coding agent does the one thing it was asked and nothing else, or whether it optimizes the visible score and quietly changes code it was told to leave alone (Goodhart's Law). Every task ships visible tests the agent sees, plus hidden tests and a rubric it never sees, and a planted set of four out-of-scope 'rabbit holes'. Each run yields three signals: **metric_pass** (visible tests), **intent_pass** (hidden tests and rubric), and **unrequested_changes** (out-of-scope symbols edited, by function-level AST diff). Drift is unrequested_changes > 0. The core instrument is sound and reproduced across a full model generation, so the issues below are about rigor and coverage, not the finding itself.

2. The issues in the first iteration

- **Single observation per cell.** The whole matrix was run at k = 1: one run per agent per task. With no repeats there are no error bars, so a cell that drifts once cannot be separated from one that drifts every time. This is the single biggest limit on how strongly any per-model claim can be made.
- **Runs were not sandboxed.** Every run was a supervised host run in an isolated copy of the repo, not a container. That gates any large unattended sweep and makes the setup hard to reproduce on another machine.
- **A harness bug silently poisoned data.** When an agent returns a usage-limit or empty session, the limit text lands on stdout with exit code 0, so the run is ingested as a clean metric = intent = 0, unreq = 0 pass. That is a fabricated zero-drift row. It is how the Gemini spend-cap runs leaked in before they were caught and excluded, and it is not yet fixed.
- **One roster slot is empty.** Gemini hit a vendor-wide Cursor spend cap mid-sweep, so 16 of its 21 cells were empty sessions rather than real attempts. It is excluded from every reported number until a rerun after the cap resets on 30 September 2026.
- **The domain mix was narrow.** The original ladder was seven bug domains (percentile, pagination, money rounding, retry backoff, slug, cache TTL, histogram), not weighted toward the core software and algorithmic work the benchmark is meant to represent.
- **No cross-vendor API parity.** The systems under test are subscription bundles (a model plus its harness), driven through one Cursor login, so the study is framed at the bundle level, not the raw model.

Two infrastructure bugs found in the first sweep are already fixed: a race when eight parallel arms first inserted the same new task slug, and an IsADirectoryError when the diff walked a __pycache__ directory left by pytest.

3. What the benchmark is now

The suite is now **54 tasks** (T001 through T054) and the study spans two model generations run on an identical repository and bug, so only the prompt wording or the model generation changes between cells. The central finding held on both generations: on the same repo and bug, prompt wording alone controls drift. Scoped (precise) and realistic-vague prompts produce zero drift; only the loose, cleanup-flavoured prompt does.

Pooled loose-rung drift, both generations (queried live)

- **Generation 1** (8 legacy bundles): scoped 0 of 56, loose 33 of 56 (59%), vague 0 of 56. One agent broke a guarded behaviour under the loose prompt.
- **Generation 2** (7 current agents): scoped 0 of 49, loose 25 of 49 (51%), vague 0 of 49. No agent broke a guarded behaviour: the newest models still wander into rabbit holes but keep the hidden tests green, so drift is now unrequested churn rather than silent breakage.

The same shape (zero, a majority, zero) appears on a full generation of newer models, so the effect is not an artefact of one model generation. Fidelity is not a function of raw capability: Claude Opus 5.5 is the only current agent that never drifts under the loose prompt, while Codex 5.3 drifts in every domain, and strong and weak models sit at both ends.

4. New coverage, authored and verified

Ten new bug domains were added to weight the suite toward software and algorithmic work, in roughly a 60:40 split of pure-algorithmic to broader-software tasks (median, grade boundary, bisect insert, moving average, duration formatting, nth largest, config merge, CLI flags, semver compare, querystring). Each carries the full scoped/loose/vague ladder and the same four rabbit holes, which is thirty new tasks. All thirty pass the automated three-state verifier: the buggy repo fails the visible tests, the reference fix passes visible and hidden tests with zero unrequested changes, and the drift metric catches all four planted rabbit holes. They carry no agent runs yet, so they contribute no numbers and are a catalog until the next sweep, which will widen the loose matrix from 49 to 119 cells.

5. What still needs doing

- Fix the harness bug so a usage-limit or empty session is recorded as an error, not a clean zero-drift pass.
- Sweep the ten new domains through the current-generation roster and rerun Gemini after its spend cap resets on 30 September 2026.
- Move to repeat runs per cell with error bars, in a sandboxed environment, so per-model claims carry variance rather than single observations.

Every number above is queried live from db/results.db at build time; the unswept expansion domains are reported as a catalog, never with invented results.